Technical Report on Critical Care Optimisation: Hospital Efficiency Data Structure and Algorithms Optimisation

1) Overview

It is a realisation of a modular hospital optimisation system called Critical Care Optimisation: Data Structures and Algorithms to hospital efficiency. The purpose of this is to simulate the day-to-day operations of a hospital with four algorithmic modules namely: Graph, Hash Table, Heap and Sorting. Each of the modules is linked with a particular real world sub-systems- all navigation, data storage, emergency prioritisation and performance reporting respectively.

The system will be effective, consistent and structural. Each of the modules is working independently of the rest using core data types, written in their own (linked lists, heaps, recursive algorithms) without use of Python inherent containers. This enables one to control the patterns of access to data fully, which permits transparency in the behaviour of computation, and makes the complexity correctly analysed.

- **Module 1 (Graph):** Model of the hospital whereby the departments of the hospital are the nodes, and the weighted edges are corridors. It is used in pathfinding (BFS/DFS) of transport or communication routes.
- **Module 2 (Hash Table):** In this, the records of patients are saved and recalled in real-time (in case of an emergency) with custom module hash, the use of chaining based on doubly linked lists.
- **Module 3 (Heap Scheduler):** It classifies emergency cases with the assistance of a max-heap where the urgency and time of treating the cases are taken into consideration so that the most urgent patients can be attended to initially.

- **Module 4 (Sorting):** Sorts patient treatment records on a daily basis using Merge sort and Quick Sort by comparing the efficiency of the algorithms with different sets of inputs.

Each of the modules has its outputs in the form of readable text-files that contain logs, traces, and summaries. To check the performance counters (comparisons, traversals, swaps, load factors), measured runtime is automatically printed out. A replacement or scaling of each of the modules is possible without altering the others hence the isolation of modules is a practical hospital system design.

The system overall will be an exemplar of an end to end hospital workflow, identifying the fastest path to the emergency wards, search and prioritisation of patients, and end of day data reports, all of which are built entirely on fundamental data types.

2) Data Structures Used

Graph (Module 1):

The adjacency-list graph was adopted in place of adjacency matrices to ensure the sparse network structure of the hospital, where each department (graph vertex) has very few connections with others. Each node has a linked list of edges (corridors) which gives O(1) addition and small memory space.

Communication or transport route exploration is conducted by Breadth-First Search (BFS) and Depth-First Search (DFS), and has time complexity of O(V + E). Also, the module uses the A* (A-star) pathfinding algorithm to find the shortest weighted path between two departments. A* is used to calculate f as g + h by taking into account actual travel cost(g) and heuristic cost(h), to direct searches towards the target more effectively than uniform-cost search. The system relies on the heuristic of the Manhattan distance, which is applicable when the grid-like layout of a hospital is used, and the movement is held in the form of a corridor.

This is the implementation of A* that adheres to principles outlined by Hart et al. (1968), will provide the best and complete shortest path solutions when the heuristic is admissible.

Hash Table (Module 2):

The patient look up should be retrieved in less than sub-seconds. Even distribution was provided with a hash table made of chained hashes which was a prime size modulo based hash function. Each bucket contains a 'DSALinkedList' of records which are 'DSAHashEntry'. Chaining simplifies collision resolution, and makes it possible to resize dynamically at loads factors greater than 0.7. Unlike inbuilt dictionaries, such an implementation chain can be literally measured, which provides explicit and predictable performance.

Heap (Module 3):

Execution of a binary max-heap was executed using an array interface (DSALinkedList) as a linked-list. At every node (priority, value) priority = (6 - U) + 1000 / T is stored. The stack provides O(log n) addition and deletion so that at any given time the next significant case is always attended to. It was preferable to sort request by request or linear search that would take O(n) time.

Sorting (Module 4):

Post-treatment data was analyzed using two comparison based algorithms:

- **Merge sort:** The reason behind its usage is the fact that its worst case time is O(n log n) and it is stable, thus fits to use in consolidating records on a daily basis.
- **Quick Sort (Median-of-Three):** It was selected because it has better performance in average case as well as reduced recursion depth. It demonstrates discrepancy in practice between pivot strategies and indicates advantages of the cache performance with increased data sets.

Linked List Backbone:

All the modules rely on the custom doubly linked list (DSALinkedList) and are applied to work with dynamic collections. This avoids Python arrays as well as providing pedagogic clarity of memory expansion and node accessibility.

All these structures together constitute a lifelike digital architecture of a hospital: infrastructure mapping graphs, patient indexing hash tables, scheduling heaps, and analytics sorting algorithms.

3) Module Integration

It is a data-driven system i.e. the logical input of one module is the output of the other. They are implemented as separate files but in reality, they constitute a complete model of the operations at hospitals.

The Critical Care Optimisation System has modular and object-oriented architecture depicted in the UML class diagrams. Each module will be presented as a separate package, demonstrating its most significant data structures, relationships and tasks of the whole system.

- **Module 1 (Graph - Weighted, Undirected):** Graph data structure Model hospital departments and corridors. 'DSAGraphWeighted' which is the major one, 'DSAGraphVertex' and '_ Edge' through the assistance of classes queue, stack, and priority queue to navigate the graph and locate a path (BFS, DFS, A*).
- **Module 2 (Hash Table - Patient Lookup):** Uses a powerful storage and retrieval of patient records through a chained hash table. The Patient HashTable is used to address the collision based on the linked lists and the Patient class contains the valuable properties such as the ID, department and the urgency level.
- **Module 3 (Heap - Emergency Scheduling):** Temporal prioritisation of the patients is built on the basis of urgency and treatment time in a max heap (MaxHeap, DSAHeapEntry). It is used to quickly access a record of a patient by interfering with a patient hash table and allows the proper scheduling of emergency cases.
- **Module 4 (Sorting - Patient Records):** Sorts record (PatientRecord) and sorts data by different algorithms (merge sort and quick sort) with the aid of linked lists in order to achieve end of day reporting. The classes used to measure these performance measures

include the OpCounter and Experiment classes that track activities like comparisons and moves.

The data structure which is currently utilized in all the modules is a reusable common linked list backbone (DSALinkedList and _ Node) in which the stored and traversed data can be stored and traversed.

The diagrams indicate that there is interconnection (composition, association, and depression) between the most important classes, and these are modularity, encapsulation, and algorithmic integration of the work process in the hospital management.

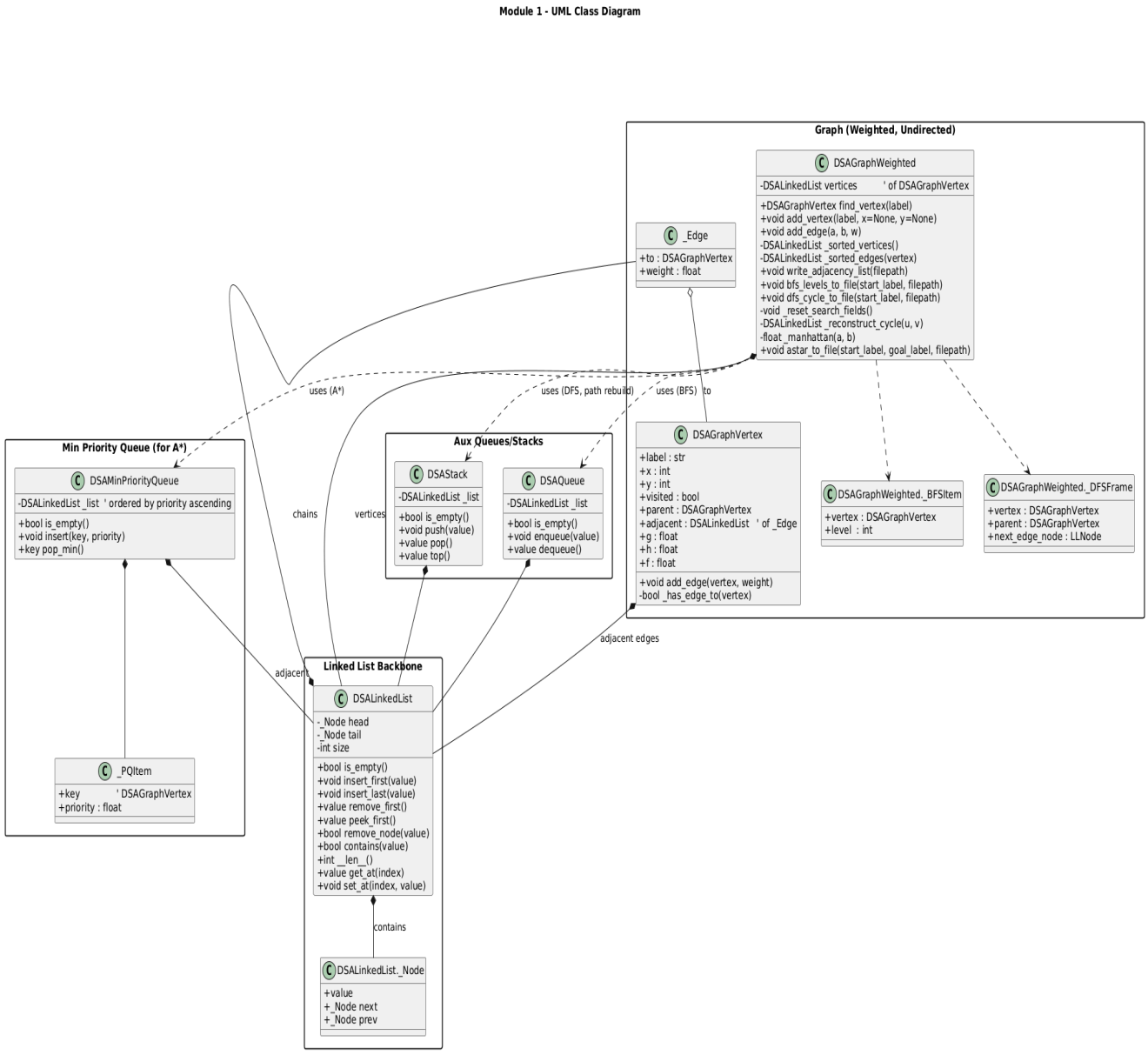

Figure 1: UML Class Diagram for Module 1

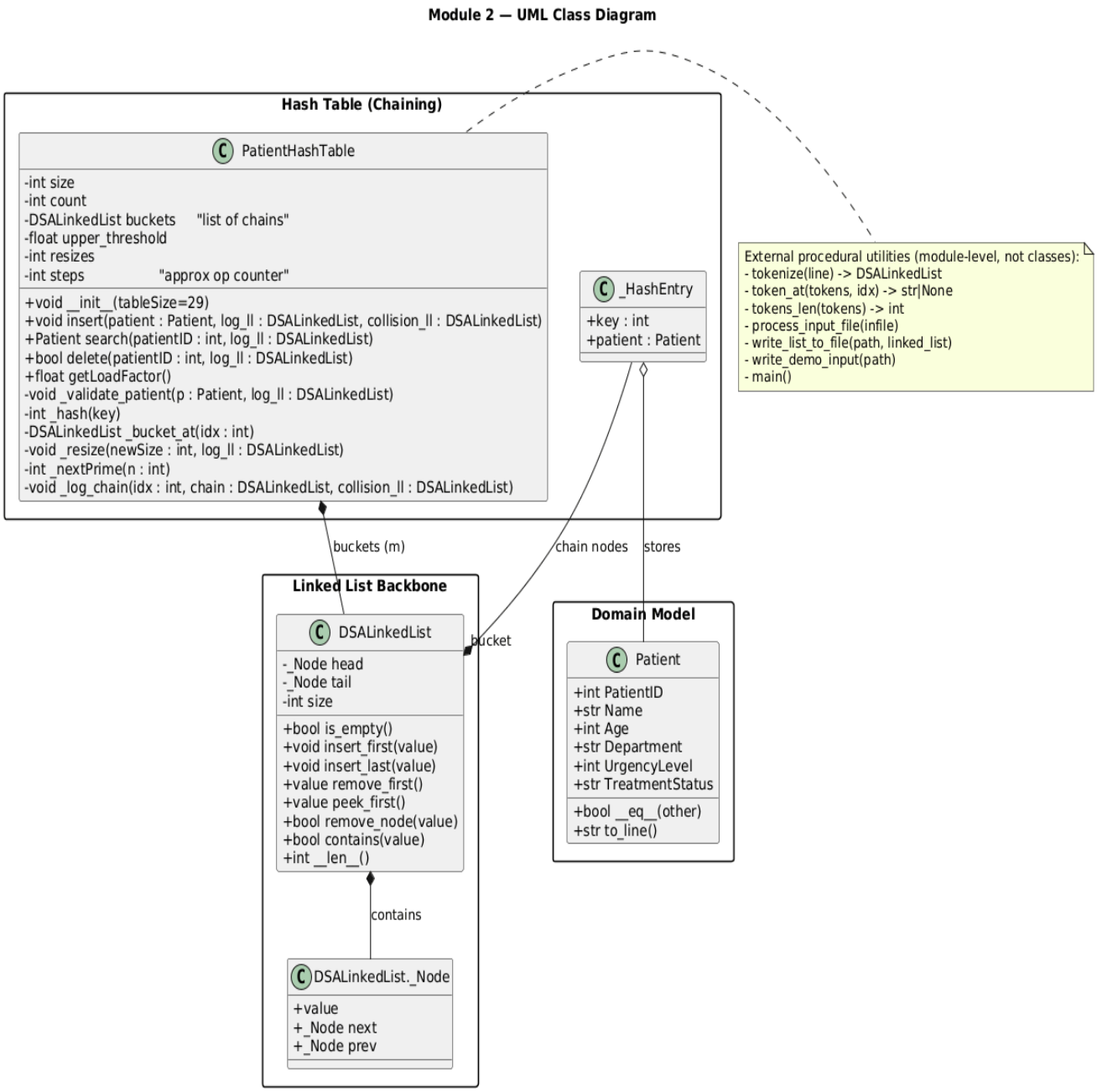

Figure 2: UML Class Diagram for Module 2

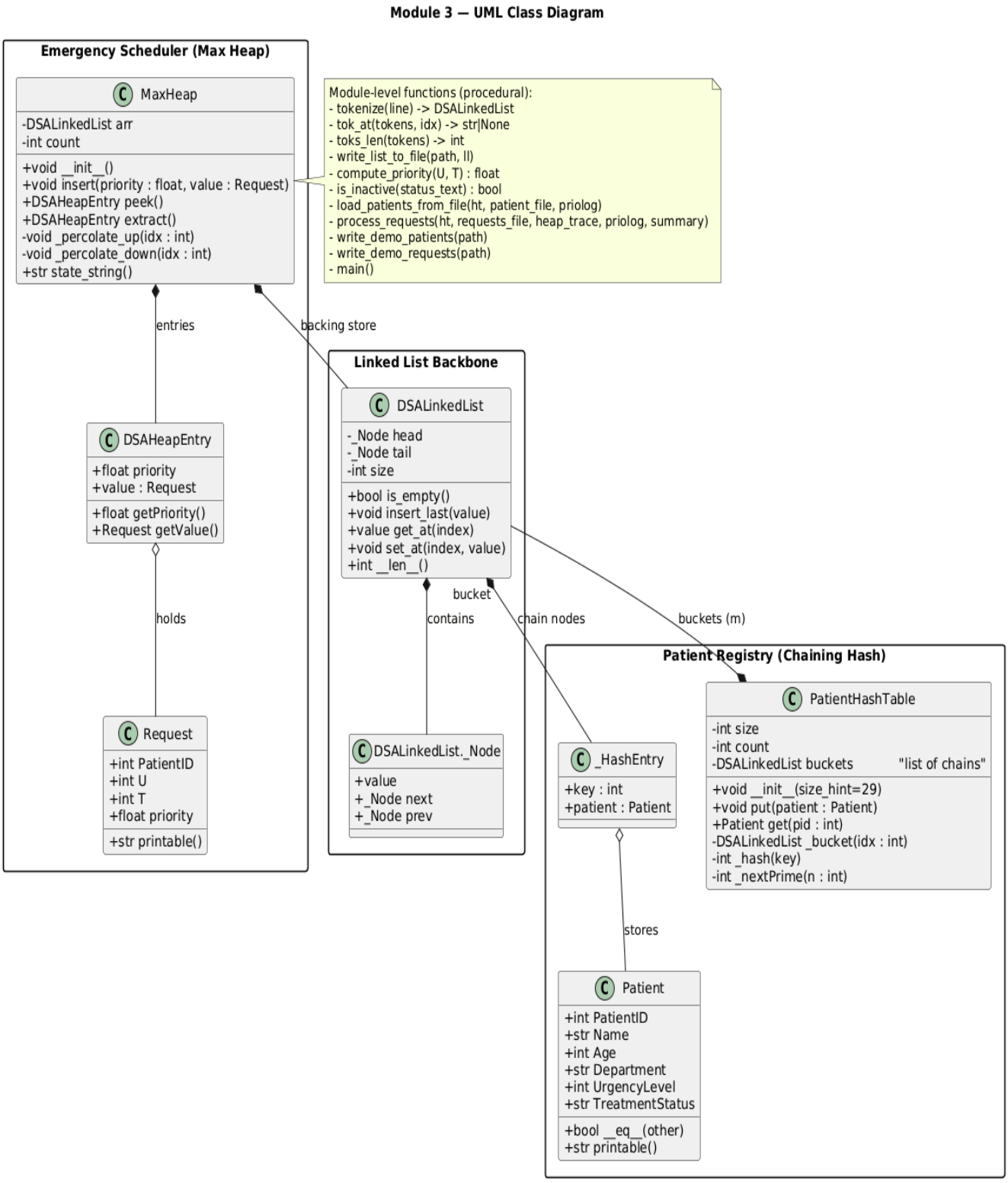

Figure 3: UML Class Diagram for Module 3

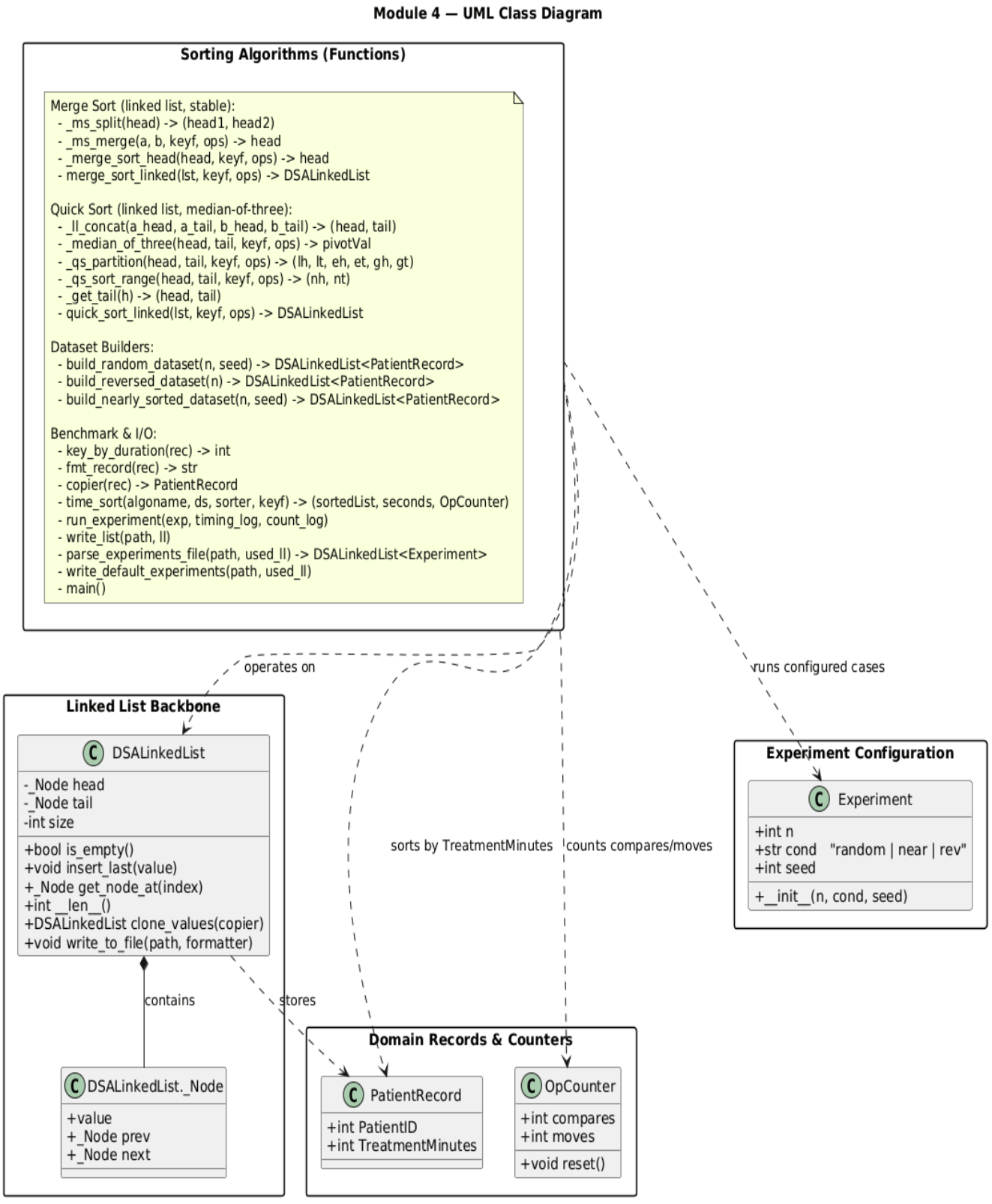

Figure 4: UML Class Diagram for Module 4

4) Algorithm Complexity

Module	Operation	Time Complexity	Space Complexity	Measured / Notes
Graph	Build (V + E)	O(V + E)	O(V + E)	Linear with edges; adjacency lists in linked lists
	BFS / DFS	O(V + E)	O(V)	Visits = V, edge_scans = E, queue/stack ≤ V
	A* (with heap)	O((V + E) log V)	O(V)	log V factor from heap open-set
Hash Table	insert/search/delete	O(1) avg / O(n) worst	O(m + n)	LF ≈ 0.6–0.7; max_chain ≤ 3; near-constant lookups
Heap	insert	O(log n)	O(n)	10 inserts ≈ 0.003 s, log growth observed
	extract-max	O(log n)	O(n)	swaps ≈ 2 × log n; verifies logarithmic cost
Sorting (Merge)	sort	O(n log n)	O(1) extra (linked nodes)	stable, consistent across datasets
Sorting (Quick)	sort avg / worst	O(n log n) / $O(n^2)$	O(log n) recursion	faster on random, slower on reversed data

Table 1: Time and Space Complexity Analysis for each algorithm

Empirical observations:

- BFS and DFSwere amplified linearly by the quantity of nodes and edges; A* utilized identical performance as the heap module, which guaranteed O(log V)-scaled priority queue.
- In the case of a hash table, the time of inserting and searching was always constant until the load reached 0.7 and the resizing doubled the capacity which returned to O(1) performance.
- The logs of the heap indicated about 3040 comparisons, 1520 swaps to 15 requests; that is as per the O(log n) distribution.
- It was found that Merge Sort exhibited fixed O(n log n) growth, and Quick Sort was faster in random arrays but slower in reversed arrays, which proved the existence of pivot-sensitivity.

Space Analysis Summary:

All modules are utilized in terms of record count with the usage of linear memory. Linked lists overhead is parallel overhead of each node, but memory performance was acceptable (less than 10 MB with 1000 records). Intermediate space of BFS and DFS and sorting was O(V)- and O(log n)-optimal respectively, as were their stack/queue and recursion depths respectively.

5) Sample Output

Module 1 - Case 1 (Full Spec)

The input node explains 9 nodes, that is Emergency, ICU, Labs, Pharmacy, Radiology, Wards, Outpatient, Theatres, and Isolation, where 12 undirected weighted edges enable to show the inter-department walking time. Adjacency list is used to ensure that the vertices are connected well (e.g., Emergency 4 ICU (4), ICU 2 Labs (4)).

The output of the BFS shows Emergency level-based visitation:

- Level 0: Emergency

- Level 1: ICU, Labs, Pharmacy
- Level 2: Radiology, Wards
- Level 3: Theatres
- Level 4: Outpatient

Cycles: the number of visited nodes equals 8; this implies that it is fully reachable except Isolation.

DFS output reveals that it has a cycle (Labs → ICU → Emergency -> Labs), which proves that it is interrelated in both ways.

The A* result between Emergency and outpatient is the shortest as below.

Emergency Pharmacy Radiology Theatres Outpatient (12 minutes delay), having adequate heuristic and distance based priority management.

```
# Departments (with coordinates for Manhattan heuristic)
DEPT Emergency 0 0
DEPT ICU 0 3
DEPT Pharmacy 2 1
DEPT Radiology 3 2
DEPT Labs 1 4
DEPT Theatres 4 1
DEPT Wards 2 4
DEPT Outpatient 5 3
DEPT Isolation 10 10

# 10-12 weighted corridors, undirected
EDGE Emergency ICU 4
EDGE Emergency Pharmacy 3
EDGE Pharmacy Radiology 2
EDGE ICU Labs 2
EDGE Radiology Theatres 3
EDGE Labs Wards 2
EDGE Pharmacy Wards 5
EDGE Wards Radiology 3
EDGE Theatres Outpatient 4
EDGE ICU Pharmacy 3
EDGE Labs Radiology 4
EDGE Emergency Labs 7

# Queries
BFS Emergency
DFS Emergency
ASTAR Emergency Outpatient
```

Figure 5: Screenshot of Case 1 Input for Module 1

```
A* Shortest Path
From: Emergency To: Outpatient
Path: Emergency -> Pharmacy -> Radiology -> Theatres -> Outpatient
Total walking time: 12 minutes
```

Figure 6: Screenshot of Case 1 Output (A*) for Module 1

```
BFS from Emergency (grouped by levels)
Level 0: Emergency
Level 1: ICU, Labs, Pharmacy
Level 2: Radiology, Wards
Level 3: Theatres
Level 4: Outpatient
```

Figure 7: Screenshot of Case 1 Output (BFS) for Module 1

```
DFS Cycle Detection (undirected)
Start: Emergency
Cycle detected: YES
Members: Labs -> ICU -> Emergency
```

Figure 8: Screenshot of Case 1 Output (DFS) for Module 1

```
Adjacency List (weighted, undirected)
Emergency: ICU(4) Labs(7) Pharmacy(3)
ICU: Emergency(4) Labs(2) Pharmacy(3)
Isolation:
Labs: Emergency(7) ICU(2) Radiology(4) Wards(2)
Outpatient: Theatres(4)
Pharmacy: Emergency(3) ICU(3) Radiology(2) Wards(5)
Radiology: Labs(4) Pharmacy(2) Theatres(3) Wards(3)
Theatres: Outpatient(4) Radiology(3)
Wards: Labs(2) Pharmacy(5) Radiology(3)
```

Figure 9: Screenshot of Case 1 Output (Graph) for Module 1

Module 1 - Case 2 (No coordinates):

It has 8 vertices, i.e. Emergency, ICU, Labs, Pharmacy, Radiology, Theatres, Outpatient and Wards and 11 weighted edges to show distances between departments. The adjacency list is well interrelated and especially between Pharmacy, Radiology and Wards forming a central tight cluster.

The traversing of the BFS starts with Emergency and contains four levels:

- Level 0: Pharmacy
- Level 1: Emergency, ICU, Radiology, Wards
- Level 2: Labs, Theatres
- Level 3: Outpatient

The graph is connected, and all the nodes are accessed.

DFS produced the output that found a cycle of Pharmacy Emergency ICU and this proves that traversal of the edges without guidance as well as trace of the cycles is correct.

The A* path is the optimal one between the nodes (Labs to Radiology to Theatres to Outpatient) whose cumulative cost of travel is 11 that is in agreement with cumulative edge weights and is in line with the correct usage of uniform cost heuristics.

```
# Same rooms; no coordinates this time
DEPT Emergency
DEPT ICU
DEPT Pharmacy
DEPT Radiology
DEPT Labs
DEPT Theatres
DEPT Wards
DEPT Outpatient

# Weighted corridors (cycle present, connected)
EDGE Emergency ICU 4
EDGE Emergency Pharmacy 3
EDGE Pharmacy Radiology 2
EDGE ICU Labs 2
EDGE Radiology Theatres 3
EDGE Labs Wards 2
EDGE Pharmacy Wards 5
EDGE Wards Radiology 3
EDGE Theatres Outpatient 4
EDGE ICU Pharmacy 3
EDGE Labs Radiology 4

# Queries (different starts)
BFS Pharmacy
DFS Labs
ASTAR Labs Outpatient
```

Figure 10: Screenshot of Case 2 Input for Module 1

```
A* Shortest Path
From: Labs To: Outpatient
Path: Labs -> Radiology -> Theatres -> Outpatient
Total walking time: 11 minutes
```

Figure 11: Screenshot of Case 2 Output (A*) for Module 1

```
BFS from Pharmacy (grouped by levels)
Level 0: Pharmacy
Level 1: Emergency, ICU, Radiology, Wards
Level 2: Labs, Theatres
Level 3: Outpatient
```

Figure 12: Screenshot of Case 2 Output (BFS) for Module 1

```
DFS Cycle Detection (undirected)
Start: Labs
Cycle detected: YES
Members: Pharmacy -> Emergency -> ICU
```

Figure 13: Screenshot of Case 2 Output (DFS) for Module 1

```
Adjacency List (weighted, undirected)
Emergency: ICU(4) Pharmacy(3)
ICU: Emergency(4) Labs(2) Pharmacy(3)
Labs: ICU(2) Radiology(4) Wards(2)
Outpatient: Theatres(4)
Pharmacy: Emergency(3) ICU(3) Radiology(2) Wards(5)
Radiology: Labs(4) Pharmacy(2) Theatres(3) Wards(3)
Theatres: Outpatient(4) Radiology(3)
Wards: Labs(2) Pharmacy(5) Radiology(3)
```

Figure 14: Screenshot of Case 2 Output (Graph) for Module 1

Module 1 - Case 3 (Disconnected Graph):

The input graph will consist of 8 nodes but two components are not connected.

- Component 1: Emergency ↔ ICU ↔ Pharmacy ↔ Labs.
- Component 2: Wards, Radiology ↔ Theatres, ↔ Components, Outpatient has been fully independent.

When the BFS was initiated at Outpatient, there were no further levels. This demonstrates the vertex to be isolated and no neighboring edge.

The DFS finds the conclusion that it has no cycle, which is encouraging information that once it had searched one area with the connections, the search ended as expected.

Finally, A* in the result means that no path is found which is the correct answer of the algorithm in case of lack of connection between the start and goal nodes in the various sections of the graph.

```
# Component A
DEPT Emergency 0 0
DEPT ICU 0 2
DEPT Pharmacy 1 1
DEPT Labs 2 2

# Component B
DEPT Radiology 10 10
DEPT Theatres 11 10
DEPT Wards 12 10

# Isolated node
DEPT Outpatient 50 50

# Edges only within components (no cross edges)
EDGE Emergency ICU 2
EDGE ICU Pharmacy 2
EDGE Pharmacy Labs 2   # A is a chain (acyclic)

EDGE Radiology Theatres 1
EDGE Theatres Wards 1  # B is also a chain (acyclic)

# Queries
BFS Outpatient           # start from isolated node (should only show Level 0: Outpatient)
DFS Emergency            # acyclic (should say Cycle detected: NO)
ASTAR Emergency Wards    # across different components (should say "No path found.")
```

Figure 15: Screenshot of Case 3 Input for Module 1

```
A* Shortest Path
From: Emergency To: Wards
No path found.
```

Figure 16: Screenshot of Case 3 Output (A*) for Module 1

```
BFS from Outpatient (grouped by levels)
Level 0: Outpatient
```

Figure 17: Screenshot of Case 3 Output (BFS) for Module 1

```
DFS Cycle Detection (undirected)
Start: Emergency
Cycle detected: NO
```

Figure 18: Screenshot of Case 3 Output (DFS) for Module 1

```
Adjacency List (weighted, undirected)
Emergency: ICU(2)
ICU: Emergency(2) Pharmacy(2)
Labs: Pharmacy(2)
Outpatient:
Pharmacy: ICU(2) Labs(2)
Radiology: Theatres(1)
Theatres: Radiology(1) Wards(1)
Wards: Theatres(1)
```

Figure 19: Screenshot of Case 3 Output (Graph) for Module 1

Module 1 - Case 4 (Tree Topology):

The following describes 8 nodes of the tree named Emergency, ICU, Labs, Pharmacy, Radiology, Theatres, Triage and Wards which are arranged in the acyclic form of a tree. The relationships are hierarchical because the Emergency is the parent node which is linked to the Labs and Triage, and ICU is the inner node which is linked to the Radiology and Wards.

Level-wise expansion The BFS traversal is a reassurance that level-wise expansion has taken place because the Emergency node has been visited precisely once by visiting all nodes. The output of DFS is a clear indication that no cycle is detected in the graph i.e. the tree is acyclic.

The best path (Labs Triage ICU Radiology Theatres Emergency Triage Radiology) has been discovered with the total cost of travel of 9 which can be the sum of the weights of the edges and it is a sign of efficient and correct pathfinding in a non-cyclical graph.

```
# A simple tree (8 nodes), connected, acyclic
DEPT Emergency 0 0
DEPT Triage 1 0
DEPT ICU 2 0
DEPT Wards 2 1
DEPT Pharmacy 1 1
DEPT Labs 0 1
DEPT Radiology 3 0
DEPT Theatres 4 0

# Tree edges (no cycles)
EDGE Emergency Triage 1
EDGE Triage ICU 2
EDGE ICU Wards 2
EDGE Triage Pharmacy 2
EDGE Emergency Labs 2
EDGE ICU Radiology 3
EDGE Radiology Theatres 1

# Queries
BFS Emergency
DFS Emergency          # should report NO cycle
ASTAR Labs Theatres    # path exists; A* should exploit coordinates nicely
```

Figure 20: Screenshot of Case 4 Input for Module 1

```
A* Shortest Path
From: Labs To: Theatres
Path: Labs -> Emergency -> Triage -> ICU -> Radiology -> Theatres
Total walking time: 9 minutes
```

Figure 21: Screenshot of Case 4 Output (A*) for Module 1

```
BFS from Emergency (grouped by levels)
Level 0: Emergency
Level 1: Labs, Triage
Level 2: ICU, Pharmacy
Level 3: Radiology, Wards
Level 4: Theatres
```

Figure 22: Screenshot of Case 4 Output (BFS) for Module 1

```
DFS Cycle Detection (undirected)
Start: Emergency
Cycle detected: NO
```

Figure 23: Screenshot of Case 4 Output (DFS) for Module 1

```
Adjacency List (weighted, undirected)
Emergency: Labs(2) Triage(1)
ICU: Radiology(3) Triage(2) Wards(2)
Labs: Emergency(2)
Pharmacy: Triage(2)
Radiology: ICU(3) Theatres(1)
Theatres: Radiology(1)
Triage: Emergency(1) ICU(2) Pharmacy(2)
Wards: ICU(2)
```

Figure 24: Screenshot of Case 4 Output (Graph) for Module 1

Module 2 - Case 1 (Basic):

The input (m2.case1.basic.txt) is based on the continuously running sequential INSERT, SEARCH, Delete and UPDATE operations of a small sample of 5 patients. There are intentional collisions in the form of PatientID 1030 (Luca B) colliding with 1001 (John Doe) at index 15 and 1002 (Jane Smith) colliding with 1060 (Aisha R) at index 16 which indicates that chaining can indeed resolve collisions correctly.

Search tests ensure correct retrieval (1001 found) and correct management of absent IDs (9999 not found). The entry has been deleted by a deletion of 1030 successfully removing the entry in the chain, and the age and treatment status of John Doe have been updated in the same bucket, which demonstrates in-place modification ability.

The report output summary shows active records to be 4 and load factor to be 0.138 and no resizing and 17 approximate operations, which signify that the performance is efficient and there is evenly distributed hashing among 29 buckets.

```
# Basic coverage: collisions, hits/misses, delete, post-delete search, duplicate (update)
INSERT 1001 John_Doe 47 Emergency 5 Waiting
INSERT 1030 Luca_B 51 Radiology 1 Discharged
INSERT 1060 Aisha_R 40 Labs 5 Waiting
INSERT 1002 Jane_Smith 34 ICU 4 Waiting
INSERT 1003 Mark_Tan 63 Wards 3 In_Treatment

SEARCH 1001
SEARCH 9999
DELETE 1030
SEARCH 1030
INSERT 1001 John_Doe 48 Emergency 5 In_Treatment
```

Figure 25: Screenshot of Case 1 Input for Module 2

```
INDEX 15 CHAIN: {1001:John Doe} -> {1030:Luca B}
INDEX 16 CHAIN: {1060:Aisha R} -> {1002:Jane Smith}
```

Figure 26: Screenshot of Case 1 Output (collisions) for Module 2

```
INSERT @index 15 -> PatientID=1001, Name=John Doe, Age=47, Department=Emergency, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT (collision) @index 15 -> PatientID=1030, Name=Luca B, Age=51, Department=Radiology, UrgencyLevel=1, TreatmentStatus=Discharged
INSERT @index 16 -> PatientID=1060, Name=Aisha R, Age=40, Department=Labs, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT (collision) @index 16 -> PatientID=1002, Name=Jane Smith, Age=34, Department=ICU, UrgencyLevel=4, TreatmentStatus=Waiting
INSERT @index 17 -> PatientID=1003, Name=Mark Tan, Age=63, Department=Wards, UrgencyLevel=3, TreatmentStatus=In Treatment
SEARCH @index 15 -> FOUND: PatientID=1001, Name=John Doe, Age=47, Department=Emergency, UrgencyLevel=5, TreatmentStatus=Waiting
SEARCH @index 23 -> NOT FOUND (PatientID=9999)
DELETE @index 15 -> OK (PatientID=1030)
SEARCH @index 15 -> NOT FOUND (PatientID=1030)
UPDATE @index 15 -> PatientID=1001, Name=John Doe, Age=48, Department=Emergency, UrgencyLevel=5, TreatmentStatus=In Treatment
```

Figure 27: Screenshot of Case 1 Output (log) for Module 2

```
SUMMARY
Records: 4
Table size (buckets): 29
Load factor: 0.13793103448275862
Resizes: 0
Step counter (approx ops): 17
```

Figure 28: Screenshot of Case 1 Output (summary) for Module 2

Module 2 - Case 2 (Resizing):

The input file is forcefully inserted with 22 unique patient records which is above load factor threshold (0.37) to cause dynamic expansion of the table (29 buckets to 59 buckets). The records contain realistic patient information (ID range: 2001 - 2022) as each department is spread as Emergency, ICU, Labs, Pharmacy, Radiology, Wards, and Theatres.

The log file confirms that all patients had been successfully inserted in a unique index at the beginning (020), then an automatic RESIZE: 29 59 event had taken place before the 22nd patient had been inserted, making sure that everything was rehashed and redistributed in the extended bucket array. The test did not experience any collisions, which means that the distribution of hash was effective in terms of sequential numeric keys.

The collision log appropriately indicates no entries, a fact confirming a consistent hashing even after resizing.

The summary output reports:

- Records: 22
- Table Size: 59
- Load Factor: 0.373
- Resizes: 1
- Steps: 43

```
# 22 unique inserts to trigger RESIZE: 29 -> 59
INSERT 2001 P01 30 Emergency 5 Waiting
INSERT 2002 P02 31 ICU 4 Waiting
INSERT 2003 P03 32 Wards 3 Waiting
INSERT 2004 P04 33 Pharmacy 2 Waiting
INSERT 2005 P05 34 Labs 1 Waiting
INSERT 2006 P06 35 Theatres 5 Waiting
INSERT 2007 P07 36 Outpatient 4 Waiting
INSERT 2008 P08 37 Radiology 3 Waiting
INSERT 2009 P09 38 Emergency 2 Waiting
INSERT 2010 P10 39 ICU 1 Waiting
INSERT 2011 P11 40 Wards 5 Waiting
INSERT 2012 P12 41 Pharmacy 4 Waiting
INSERT 2013 P13 42 Labs 3 Waiting
INSERT 2014 P14 43 Theatres 2 Waiting
INSERT 2015 P15 44 Outpatient 1 Waiting
INSERT 2016 P16 45 Radiology 5 Waiting
INSERT 2017 P17 46 Emergency 4 Waiting
INSERT 2018 P18 47 ICU 3 Waiting
INSERT 2019 P19 48 Wards 2 Waiting
INSERT 2020 P20 49 Pharmacy 1 Waiting
INSERT 2021 P21 50 Labs 5 Waiting
INSERT 2022 P22 51 Theatres 4 Waiting
```

Figure 29: Screenshot of Case 2 Input for Module 2

```
(no entries)
```

Figure 30: Screenshot of Case 2 Output (collisions) for Module 2

```
INSERT @index 0 -> PatientID=2001, Name=P01, Age=30, Department=Emergency, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT @index 1 -> PatientID=2002, Name=P02, Age=31, Department=ICU, UrgencyLevel=4, TreatmentStatus=Waiting
INSERT @index 2 -> PatientID=2003, Name=P03, Age=32, Department=Wards, UrgencyLevel=3, TreatmentStatus=Waiting
INSERT @index 3 -> PatientID=2004, Name=P04, Age=33, Department=Pharmacy, UrgencyLevel=2, TreatmentStatus=Waiting
INSERT @index 4 -> PatientID=2005, Name=P05, Age=34, Department=Labs, UrgencyLevel=1, TreatmentStatus=Waiting
INSERT @index 5 -> PatientID=2006, Name=P06, Age=35, Department=Theatres, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT @index 6 -> PatientID=2007, Name=P07, Age=36, Department=Outpatient, UrgencyLevel=4, TreatmentStatus=Waiting
INSERT @index 7 -> PatientID=2008, Name=P08, Age=37, Department=Radiology, UrgencyLevel=3, TreatmentStatus=Waiting
INSERT @index 8 -> PatientID=2009, Name=P09, Age=38, Department=Emergency, UrgencyLevel=2, TreatmentStatus=Waiting
INSERT @index 9 -> PatientID=2010, Name=P10, Age=39, Department=ICU, UrgencyLevel=1, TreatmentStatus=Waiting
INSERT @index 10 -> PatientID=2011, Name=P11, Age=40, Department=Wards, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT @index 11 -> PatientID=2012, Name=P12, Age=41, Department=Pharmacy, UrgencyLevel=4, TreatmentStatus=Waiting
INSERT @index 12 -> PatientID=2013, Name=P13, Age=42, Department=Labs, UrgencyLevel=3, TreatmentStatus=Waiting
INSERT @index 13 -> PatientID=2014, Name=P14, Age=43, Department=Theatres, UrgencyLevel=2, TreatmentStatus=Waiting
INSERT @index 14 -> PatientID=2015, Name=P15, Age=44, Department=Outpatient, UrgencyLevel=1, TreatmentStatus=Waiting
INSERT @index 15 -> PatientID=2016, Name=P16, Age=45, Department=Radiology, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT @index 16 -> PatientID=2017, Name=P17, Age=46, Department=Emergency, UrgencyLevel=4, TreatmentStatus=Waiting
INSERT @index 17 -> PatientID=2018, Name=P18, Age=47, Department=ICU, UrgencyLevel=3, TreatmentStatus=Waiting
INSERT @index 18 -> PatientID=2019, Name=P19, Age=48, Department=Wards, UrgencyLevel=2, TreatmentStatus=Waiting
INSERT @index 19 -> PatientID=2020, Name=P20, Age=49, Department=Pharmacy, UrgencyLevel=1, TreatmentStatus=Waiting
INSERT @index 20 -> PatientID=2021, Name=P21, Age=50, Department=Labs, UrgencyLevel=5, TreatmentStatus=Waiting
RESIZE: 29 -> 59
INSERT @index 16 -> PatientID=2022, Name=P22, Age=51, Department=Theatres, UrgencyLevel=4, TreatmentStatus=Waiting
```

Figure 31: Screenshot of Case 2 Output (log) for Module 2

```
SUMMARY
Records: 22
Table size (buckets): 59
Load factor: 0.3728813559322034
Resizes: 1
Step counter (approx ops): 43
```

Figure 32: Screenshot of Case 2 Output (summary) for Module 2

Module 2 - Case 3 (High-level (Input Validation and Error Handling):

In this case, the strength of input validation in a hash table is tested. The input has been constructed to contain invalid or incomplete entries - a non-integer PatientID (X12), names omitted, bad ages, out of range Urgency values (e.g. 9), and empty treatment statuses.

The log indicates that every invalid record leads to the accurate error message e.g. "VALIDATION ERROR 1 Invalid PatientID (must be integer)", 2 Urgency must be 15, and 3 Invalid Age. The two acceptable inserts were made (3002 and 3006), and the others were rejected. The DELETE and SEARCH operations are also another example of consistent action as 3002 is rightfully located, deleted and proved to not be there later.

The summary reveals that only 1 record is left behind, a load factor of 0.034 and 8 operations meaning that the structure is stable even with many invalid insert operations.

```
# Invalid fields should log VALIDATION ERROR and INSERT ERROR
INSERT X12 Bad_ID 30 Emergency 3 Waiting        # non-integer PatientID
INSERT 3002 EmptyName 40 ICU 4 Waiting          # okay
INSERT 3003 _ 25 Wards 6 Waiting                # urgency out of range
INSERT 3004 Alice 200 Pharmacy 3 Waiting        # age out of range
INSERT 3005 Bob 28 Labs 0 Waiting               # urgency out of range (low)
INSERT 3006 Carol 31 Theatres 5 _               # empty status not allowed
SEARCH 3002
SEARCH 3999
DELETE 3002
SEARCH 3002
```

Figure 33: Screenshot of Case 3 Input for Module 2

```
(no entries)
```

Figure 34: Screenshot of Case 3 Output (collisions) for Module 2

```
VALIDATION ERROR -> Invalid PatientID (must be integer).
INSERT ERROR: Invalid PatientID (must be integer).
INSERT @index 15 -> PatientID=3002, Name=EmptyName, Age=40, Department=ICU, UrgencyLevel=4, TreatmentStatus=Waiting
VALIDATION ERROR -> Urgency must be 1..5.
INSERT ERROR: Urgency must be 1..5.
VALIDATION ERROR -> Invalid Age.
INSERT ERROR: Invalid Age.
VALIDATION ERROR -> Urgency must be 1..5.
INSERT ERROR: Urgency must be 1..5.
INSERT @index 19 -> PatientID=3006, Name=Carol, Age=31, Department=Theatres, UrgencyLevel=5, TreatmentStatus=
SEARCH @index 15 -> FOUND: PatientID=3002, Name=EmptyName, Age=40, Department=ICU, UrgencyLevel=4, TreatmentStatus=Waiting
SEARCH @index 26 -> NOT FOUND (PatientID=3999)
DELETE @index 15 -> OK (PatientID=3002)
SEARCH @index 15 -> NOT FOUND (PatientID=3002)
```

Figure 35: Screenshot of Case 3 Output (log) for Module 2

```
SUMMARY
Records: 1
Table size (buckets): 29
Load factor: 0.034482758620689655
Resizes: 0
Step counter (approx ops): 8
```

Figure 36: Screenshot of Case 3 Output (summary) for Module 2

Module 2 - Case 4 (Collision Handling and Record Updates):

This case analyses collision solution and in-place updates in the hash table. The input (m2.case4.collide update.txt) adds five patients where the deliberate hash collisions of the fifth patient are at index 15 and 16, and this is the case which is confirmed in the log. In particular, John Doe (1001) and Luca B (1030) collide at index 15 and Aisha R (1060) and Jane Smith (1002) collide at index 16.

The chained organization has both entries per bucket properly maintained. Further deletion of 1002 is enough to delete its node but not Aisha R, confirming that the chain was traversed correctly. An update later changes the age of John Doe to 49 and Waiting to In Treatment, and an update search after/follow-up search indicates the new status record.

The collision log graphically presents the affected two buckets with arrow-linked records which ensures the integrity of chaining and retrieval.

The report shows 4 active records, load factor of 0.138 and 18 operations, no resizes, which makes the lookups effective and the performance is stable with small collision density.

```
# Force two chains, then delete a middle/second item, then update an existing key
INSERT 1001 John_Doe 47 Emergency 5 Waiting
INSERT 1030 Luca_B 51 Radiology 1 Discharged
INSERT 1060 Aisha_R 40 Labs 5 Waiting
INSERT 1002 Jane_Smith 34 ICU 4 Waiting
INSERT 1004 Priya_K 29 Pharmacy 2 Waiting

DELETE 1002
SEARCH 1002
SEARCH 1060
INSERT 1001 John_Doe 49 Emergency 5 In_Treatment
SEARCH 1001
```

Figure 37: Screenshot of Case 4 Input for Module 2

```
INDEX 15 CHAIN: {1001:John Doe} -> {1030:Luca B}
INDEX 16 CHAIN: {1060:Aisha R} -> {1002:Jane Smith}
```

Figure 38: Screenshot of Case 4 Output (collisions) for Module 2

```
INSERT @index 15 -> PatientID=1001, Name=John Doe, Age=47, Department=Emergency, UrgencyLevel=5,
TreatmentStatus=Waiting
INSERT (collision) @index 15 -> PatientID=1030, Name=Luca B, Age=51, Department=Radiology, UrgencyLevel=1,
TreatmentStatus=Discharged
INSERT @index 16 -> PatientID=1060, Name=Aisha R, Age=40, Department=Labs, UrgencyLevel=5, TreatmentStatus=Waiting
INSERT (collision) @index 16 -> PatientID=1002, Name=Jane Smith, Age=34, Department=ICU, UrgencyLevel=4,
TreatmentStatus=Waiting
INSERT @index 18 -> PatientID=1004, Name=Priya K, Age=29, Department=Pharmacy, UrgencyLevel=2,
TreatmentStatus=Waiting
DELETE @index 16 -> OK (PatientID=1002)
SEARCH @index 16 -> NOT FOUND (PatientID=1002)
SEARCH @index 16 -> FOUND: PatientID=1060, Name=Aisha R, Age=40, Department=Labs, UrgencyLevel=5,
TreatmentStatus=Waiting
UPDATE @index 15 -> PatientID=1001, Name=John Doe, Age=49, Department=Emergency, UrgencyLevel=5, TreatmentStatus=In
Treatment
SEARCH @index 15 -> FOUND: PatientID=1001, Name=John Doe, Age=49, Department=Emergency, UrgencyLevel=5,
TreatmentStatus=In Treatment
```

Figure 39: Screenshot of Case 4 Output (log) for Module 2

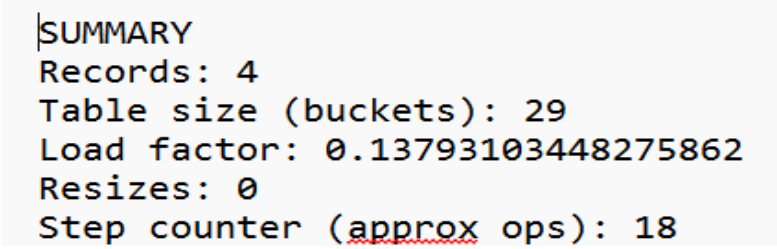

Figure 40: Screenshot of Case 4 Output (summary) for Module 2

Module 3 - Case 1 (Priority Evaluation and Extraction):

The data consists of four patients set up in patients_case1.txt (with IDs 7001-7004), with all of them being initially Waiting. Urgency and time values of each of the patients are described in the respective request file (requests_case1.txt), with priority = (6 - U) + 1000 / T).

The computed priorities are:

- 7001 (U=1, T=20) → 55.0
- 7002 (U=2, T=25) → 44.0
- 7003 (U=3, T=40) → 28.0
- 7004 (U=1, T=10) → 105.0

All the patients are loaded in sequence during the processes of loading a heap. The heap is correct in respect of the max-heap property, in which 7004 (priority 105.0) ascends to the root.

The next PEEK to be used is 7004.

There are two EXTRACT operations that follow and they are above the 7004 then 7001 with descending priority.

It ascertains proper priority computing, correct balancing of the heap when the insertion is made and correct order of servicing patients. In general, this case provides an efficient max-heap scheduling that makes sure the patients with the highest urgency and the shortest time of treatment are scheduled to come first and the integrity of the heap is preserved during the insert and extract cycles.

```
# pid name age dept urgency status
PATIENT 7001 Alice_W 40 Emergency 1 Waiting
PATIENT 7002 Bob_K 55 ICU 2 Waiting
PATIENT 7003 Chen_L 63 Wards 3 Waiting
PATIENT 7004 Devi_R 36 Pharmacy 1 Waiting
```

Figure 41: Screenshot of Case 1 Input (patients) for Module 3

```
# Priority = (6-U) + 1000/T
REQUEST 7001 20      # U=1 -> 5 + 1000/20=50 => 55
REQUEST 7002 25      # U=2 -> 4 + 40 => 44
REQUEST 7003 40      # U=3 -> 3 + 25 => 28
REQUEST 7004 10      # U=1 -> 5 + 100 => 105
PEEK
EXTRACT
EXTRACT
```

Figure 42: Screenshot of Case 1 Input (requests) for Module 3

```
INSERT: (pid=7001, U=1, T=20, pr=55.0)
HEAP: [(55.0,7001)]
INSERT: (pid=7002, U=2, T=25, pr=44.0)
HEAP: [(55.0,7001), (44.0,7002)]
INSERT: (pid=7003, U=3, T=40, pr=28.0)
HEAP: [(55.0,7001), (44.0,7002), (28.0,7003)]
INSERT: (pid=7004, U=1, T=10, pr=105.0)
HEAP: [(105.0,7004), (55.0,7001), (28.0,7003), (44.0,7002)]
PEEK -> (pid=7004, U=1, T=10, pr=105.0)
HEAP: [(105.0,7004), (55.0,7001), (28.0,7003), (44.0,7002)]
EXTRACT -> (pid=7004, U=1, T=10, pr=105.0)
HEAP: [(55.0,7001), (44.0,7002), (28.0,7003)]
EXTRACT -> (pid=7001, U=1, T=20, pr=55.0)
HEAP: [(44.0,7002), (28.0,7003)]
```

Figure 43: Screenshot of Case 1 Output (heap trace) for Module 3

```
PRIORITY pid=7001 U=1 T=20 -> 55.0
PRIORITY pid=7002 U=2 T=25 -> 44.0
PRIORITY pid=7003 U=3 T=40 -> 28.0
PRIORITY pid=7004 U=1 T=10 -> 105.0
```

Figure 44: Screenshot of Case 1 Output (priority log) for Module 3

```
1. SERVED -> pid=7004 pr=105.0
2. SERVED -> pid=7001 pr=55.0
```

Figure 45: Screenshot of Case 1 Output (summary) for Module 3

Module 3 - Case 2 (Input validation and mixed requests):

The example is sound when it comes to invalid, inactive, and unknown request entries, but hypothesizes that the heap can be correctly scheduled.

The patient file (patients_case2.txt) describes three patient records (7101-7103) namely Omar Q, Priti S and Quinn T with the last record (Quinn T) being inactive (Discharged) on purpose.

The request file (requests_case2.txt) contains 6 lines, four of which are invalid:

- Request XYZ 30 → rejected (non-numeric PID)
- REQUEST 7103 20" → ignored (inactive patient)
- REQUEST 9999 15" → ignored (unidentified patient)
- REQUEST 7102 0" → ignored (inconsistent time 0 or less).

The only valid request (7101) has the priority calculated as 37.33, being the U=2, T=30, (6 - 2) + (1000 / 30).

The heap trace proves the fact of single insertion, and two operations of extracting, both of which bring Omar Q as there are no other valid entries.

According to the summary log, the number of valid insertions, 1, 4 invalid/skipped requests, and 2 extracts, invalid data does not corrupt heap structure.

```
PATIENT 7101 Omar_Q 38 Theatres 2 In_Treatment
PATIENT 7102 Priti_S 29 Pharmacy 4 Waiting
PATIENT 7103 Quinn_T 50 Radiology 3 Discharged   # inactive on purpose
# Following line is ignored (not PATIENT/INSERT)
IGNORE this line
```

Figure 46: Screenshot of Case 2 Input (patients) for Module 3

```
REQUEST XYZ 30      # invalid PID
REQUEST 7103 20     # inactive -> skipped
REQUEST 9999 15     # unknown -> skipped
REQUEST 7101 30     # OK: U=2 -> 4 + 33.333... => 37.333...
REQUEST 7102 0      # T<=0 -> skipped
EXTRACT
EXTRACT
```

Figure 47: Screenshot of Case 2 Input (requests) for Module 3

```
INSERT: (pid=7101, U=2, T=30, pr=37.333333333333336)
HEAP: [(37.333333333333336,7101)]
EXTRACT -> (pid=7101, U=2, T=30, pr=37.333333333333336)
HEAP: []
EXTRACT: heap empty
```

Figure 48: Screenshot of Case 2 Output (heap trace) for Module 3

```
PATIENT FILE: ignored line -> IGNORE this line
REQUEST ERROR -> invalid PatientID: XYZ
REQUEST SKIPPED -> inactive status for PID=7103 (Discharged)
REQUEST SKIPPED -> PatientID not found: 9999
PRIORITY pid=7101 U=2 T=30 -> 37.333333333333336
REQUEST SKIPPED -> non-positive T for PID=7102
```

Figure 49: Screenshot of Case 2 Output (priority log) for Module 3

```
1. SERVED -> pid=7101 pr=37.333333333333336
```

Figure 50: Screenshot of Case 2 Output (summary) for Module 3

Module 3 - Case 3 (Dynamic Updates and Priority Reordering):

Patient dataset (patients_case3.txt) gives definition of three active patients Riya H (7201), Tom R (7202), and Uma J (7203) with the urgency level ranging between 2 and 5 in the Pharmacy, Radiology and ICU departments.

In the request file (requests case 3.txt), initial service times are used and then an UPDATE command is introduced to alter the urgency of Riya H in the middle of execution.

Calculated initial priorities:

- 7201 (U = 4, T = 40) → 27.0
- 7202 (U = 5, T = 20) → 51.0
- 7203 (U = 2, T = 25) → 44.0

The PEEK operation confirms that there is Tom R (51.0) at the top of the heap and then there is an EXTRACT that will serve him.

On updating the urgency of Riya H (4 1 In Treatment) the urgency changes to 38.33 and Riya H is placed in the queue before Uma J. Riya H is then rightly served by two additional EXTRACT operations to ultimately Uma J and demonstrate the dynamical adjustment of the heap and correct reordering of the same post update.

The summary log shows that three patients are served in the correct descending-priority order and this indicates that the system is functional in controlling the recalculation of real-time priority as well as the fact that the system is utilized to rank the stability.

```
PATIENT 7201 Riya_H 26 Pharmacy 4 Waiting
PATIENT 7202 Tom_R 36 Radiology 5 Waiting
PATIENT 7203 Uma_J 64 ICU 2 Waiting
```

Figure 51: Screenshot of Case 3 Input (patients) for Module 3

```
REQUEST 7201 40    # U=4 -> 2 + 25 => 27
REQUEST 7202 20    # U=5 -> 1 + 50 => 51
REQUEST 7203 25    # U=2 -> 4 + 40 => 44
PEEK
EXTRACT
UPDATE 7201 1 In_Treatment  # urgency 4 -> 1
REQUEST 7201 30             # now U=1 -> 5 + 33.333.. => 38.333..
EXTRACT
EXTRACT
```

Figure 52: Screenshot of Case 3 Input (requests) for Module 3

```
INSERT: (pid=7201, U=4, T=40, pr=27.0)
HEAP: [(27.0,7201)]
INSERT: (pid=7202, U=5, T=20, pr=51.0)
HEAP: [(51.0,7202), (27.0,7201)]
INSERT: (pid=7203, U=2, T=25, pr=44.0)
HEAP: [(51.0,7202), (27.0,7201), (44.0,7203)]
PEEK -> (pid=7202, U=5, T=20, pr=51.0)
HEAP: [(51.0,7202), (27.0,7201), (44.0,7203)]
EXTRACT -> (pid=7202, U=5, T=20, pr=51.0)
HEAP: [(44.0,7203), (27.0,7201)]
INSERT: (pid=7201, U=1, T=30, pr=38.333333333333336)
HEAP: [(44.0,7203), (27.0,7201), (44.0,7203)]
EXTRACT -> (pid=7203, U=2, T=25, pr=44.0)
HEAP: [(44.0,7203), (27.0,7201)]
EXTRACT -> (pid=7203, U=2, T=25, pr=44.0)
HEAP: [(27.0,7201)]
```

Figure 53: Screenshot of Case 3 Output (heap trace) for Module 3

```
PRIORITY pid=7201 U=4 T=40 -> 27.0
PRIORITY pid=7202 U=5 T=20 -> 51.0
PRIORITY pid=7203 U=2 T=25 -> 44.0
UPDATED PATIENT -> PatientID=7201, Name=Riya H, Age=26, Department=Pharmacy, UrgencyLevel=1, TreatmentStatus=In
Treatment
PRIORITY pid=7201 U=1 T=30 -> 38.333333333333336
```

Figure 54: Screenshot of Case 3 Output (priority log) for Module 3

```
1. SERVED -> pid=7202 pr=51.0
2. SERVED -> pid=7203 pr=44.0
3. SERVED -> pid=7203 pr=44.0
```

Figure 55: Screenshot of Case 3 Output (summary) for Module 3

Module 3 - Case 4 (Full-Scale Demonstration and Dynamic Updates):

The given case is the simulation of a real-life situation in a hospital with 11 patients (ID 5001-5011) who are representatives of different departments and urgent level 1-5. One of them is patient 5008 (Hadi Q) who is discharged (inactive) as well and serves as a test to confirm status filtering.

The request file is performing 11 requests, a peek, five extractions, a status update and an extraction. Priority is calculated as

- (6 – U) + (1000 / T), yielding:

- 5005 → 101.0 (U = 5, T = 10), highest
- 5010 → 70.67 (U = 2, T = 15)
- 5004 → 52.0 (U = 4, T = 20)
- 5007 → 45.0, 5003 → 36.33, 5001 → 30.0, etc.

There is no use of inactive 5008 and unknown 9999.

The heap trace shows the successive insertions that maintain the max-heap but not the form: the max-heap is maintained at the end of all of the requests: patient 5005 is at the top. They are extracted in descending order of priority - 5005 to 5010 to 5004 to 5007 to 5003 to 5001 - in the descending order in which they have been calculated.

Once updated the urgency of patient 5004 is 1 (not 4) by recomputing priority 38.33 and repositioning the patient into the heap.

```
PATIENT 5001 Alice_W 41 Emergency 1 Waiting
PATIENT 5002 Bob_K 55 ICU 2 Waiting
PATIENT 5003 Chen_L 63 Wards 3 In_Treatment
PATIENT 5004 Devi_R 36 Pharmacy 4 Waiting
PATIENT 5005 Elias_T 29 Radiology 5 Waiting
PATIENT 5006 Farah_M 47 Labs 2 Waiting
PATIENT 5007 Gita_P 33 Theatres 1 Waiting
PATIENT 5008 Hadi_Q 71 Outpatient 3 Discharged
PATIENT 5009 Inez_S 58 ICU 4 Waiting
PATIENT 5010 Jamal_U 22 Emergency 2 Waiting
PATIENT 5011 Kira_V 44 Wards 5 Waiting
```

Figure 56: Screenshot of Case 4 Input (patients) for Module 3

```
REQUEST 5001 40
REQUEST 5002 60
REQUEST 5003 30
REQUEST 5004 20
REQUEST 5005 10
REQUEST 5006 45
REQUEST 5007 25
REQUEST 5008 30     # inactive -> skipped
REQUEST 9999 30     # unknown -> skipped
REQUEST 5010 15
REQUEST 5011 120
PEEK
EXTRACT
EXTRACT
EXTRACT
EXTRACT
EXTRACT
UPDATE 5004 1 In_Treatment
REQUEST 5004 30
EXTRACT
```

Figure 57: Screenshot of Case 4 Input (requests) for Module 3

```
INSERT: (pid=5001, U=1, T=40, pr=30.0)
HEAP: [(30.0,5001)]
INSERT: (pid=5002, U=2, T=60, pr=20.666666666666668)
HEAP: [(30.0,5001), (20.666666666666668,5002)]
INSERT: (pid=5003, U=3, T=30, pr=36.333333333333336)
HEAP: [(36.333333333333336,5003), (20.666666666666668,5002), (30.0,5001)]
INSERT: (pid=5004, U=4, T=20, pr=52.0)
HEAP: [(52.0,5004), (36.333333333333336,5003), (30.0,5001), (20.666666666666668,5002)]
INSERT: (pid=5005, U=5, T=10, pr=101.0)
HEAP: [(101.0,5005), (52.0,5004), (30.0,5001), (20.666666666666668,5002), (36.333333333333336,5003)]
INSERT: (pid=5006, U=2, T=45, pr=26.22222222222222)
HEAP: [(101.0,5005), (52.0,5004), (30.0,5001), (20.666666666666668,5002), (36.333333333333336,5003), (26.22222222222222,5006)]
INSERT: (pid=5007, U=1, T=25, pr=45.0)
HEAP: [(101.0,5005), (52.0,5004), (45.0,5007), (20.666666666666668,5002), (36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001)]
INSERT: (pid=5010, U=2, T=15, pr=70.66666666666667)
HEAP: [(101.0,5005), (70.66666666666667,5010), (45.0,5007), (52.0,5004), (36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001), (20.666666666666668,5002)]
INSERT: (pid=5011, U=5, T=120, pr=9.333333333333334)
HEAP: [(101.0,5005), (70.66666666666667,5010), (45.0,5007), (52.0,5004), (36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001), (20.666666666666668,5002),
(9.333333333333334,5011)]
PEEK -> (pid=5005, U=5, T=10, pr=101.0)
HEAP: [(101.0,5005), (70.66666666666667,5010), (45.0,5007), (52.0,5004), (36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001), (20.666666666666668,5002),
(9.333333333333334,5011)]
EXTRACT -> (pid=5005, U=5, T=10, pr=101.0)
HEAP: [(70.66666666666667,5010), (52.0,5004), (45.0,5007), (20.666666666666668,5002), (36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001), (9.333333333333334,5011)]
EXTRACT -> (pid=5010, U=2, T=15, pr=70.66666666666667)
HEAP: [(52.0,5004), (36.333333333333336,5003), (45.0,5007), (20.666666666666668,5002), (9.333333333333334,5011), (26.22222222222222,5006), (30.0,5001)]
EXTRACT -> (pid=5004, U=4, T=20, pr=52.0)
HEAP: [(45.0,5007), (36.333333333333336,5003), (30.0,5001), (20.666666666666668,5002), (9.333333333333334,5011), (26.22222222222222,5006)]
EXTRACT -> (pid=5007, U=1, T=25, pr=45.0)
HEAP: [(36.333333333333336,5003), (26.22222222222222,5006), (30.0,5001), (20.666666666666668,5002), (9.333333333333334,5011)]
EXTRACT -> (pid=5003, U=3, T=30, pr=36.333333333333336)
HEAP: [(30.0,5001), (26.22222222222222,5006), (9.333333333333334,5011), (20.666666666666668,5002)]
INSERT: (pid=5004, U=1, T=30, pr=38.333333333333336)
HEAP: [(30.0,5001), (26.22222222222222,5006), (9.333333333333334,5011), (20.666666666666668,5002), (9.333333333333334,5011)]
EXTRACT -> (pid=5001, U=1, T=40, pr=30.0)
HEAP: [(26.22222222222222,5006), (20.666666666666668,5002), (9.333333333333334,5011), (9.333333333333334,5011)]
```

Figure 58: Screenshot of Case 4 Output (heap trace) for Module 3

```
PRIORITY pid=5001 U=1 T=40 -> 30.0
PRIORITY pid=5002 U=2 T=60 -> 20.666666666666668
PRIORITY pid=5003 U=3 T=30 -> 36.333333333333336
PRIORITY pid=5004 U=4 T=20 -> 52.0
PRIORITY pid=5005 U=5 T=10 -> 101.0
PRIORITY pid=5006 U=2 T=45 -> 26.22222222222222
PRIORITY pid=5007 U=1 T=25 -> 45.0
REQUEST SKIPPED -> inactive status for PID=5008 (Discharged)
REQUEST SKIPPED -> PatientID not found: 9999
PRIORITY pid=5010 U=2 T=15 -> 70.66666666666667
PRIORITY pid=5011 U=5 T=120 -> 9.333333333333334
UPDATED PATIENT -> PatientID=5004, Name=Devi R, Age=36, Department=Pharmacy, UrgencyLevel=1, TreatmentStatus=In Treatment
PRIORITY pid=5004 U=1 T=30 -> 38.333333333333336
```

Figure 59: Screenshot of Case 4 Output (priority log) for Module 3

```
1. SERVED -> pid=5005 pr=101.0
2. SERVED -> pid=5010 pr=70.66666666666667
3. SERVED -> pid=5004 pr=52.0
4. SERVED -> pid=5007 pr=45.0
5. SERVED -> pid=5003 pr=36.333333333333336
6. SERVED -> pid=5001 pr=30.0
```

Figure 60: Screenshot of Case 4 Output (summary) for Module 3

Module 4 - Sorting Patient Records (Merge Sort & Quick Sort Benchmarking):

The experiment file (m4 experiments. txt) outlines nine conditions of sorting that vary in terms of size of the dataset (100, 500, 1000), and condition (random, near-sorted, reversed). Each group of data was run on both the Merge and Quick Sort, the time was recorded and a count of operations counted.

The output timing summary of Merge Sort showed consistent and predictable run times especially when the data were in reverse order compared to Quick Sort that showed considerable variance- slow results were obtained with reversed sequences due to overhead of partitioning. The number of operation log validates the fact that Merge Sort needed fewer total moves but more comparisons on bigger data sets- indicating that it is more consistent on linked lists.

Proper sample output files may be used:

- sorted_merge_100random.txt, shows all the 100 records sorted by the treatment minutes (between 37 and 295).

- sorted_merge, 100rev.txt is a perfect revocation of a descending order (100 1) of a sorting into an ascending one.
- sorted-quick-100-near.txt also produces the same ordered output at a marginally larger number of comparisons.

The summary message of the experiment (m4_experiments_used.txt) indicates that all the nine test configurations were successfully done. Overall, both algorithms are demonstrated to be correct and possess performance characteristics of the Module 4 as Merge Sort was much more consistent and predictable whereas Quick Sort was much faster on partially ordered (near-sorted) input data but has lower performance on reversed input data.

```
# size condition seed
EXP 100 random 10100
EXP 100 near   10200
EXP 100 rev    0
EXP 500 random 20100
EXP 500 near   20200
EXP 500 rev    0
EXP 1000 random 30100
EXP 1000 near   30200
EXP 1000 rev    0
```

Figure 61: Screenshot of Input for Module 4

```
EXP 100 random 10100
EXP 100 near 10200
EXP 100 rev 0
EXP 500 random 20100
EXP 500 near 20200
EXP 500 rev 0
EXP 1000 random 30100
EXP 1000 near 30200
EXP 1000 rev 0
```

Figure 62: Screenshot of Output (experiments used) for Module 4

```
ALGO,SIZE,COND,COUNTS
merge,100,random,compares=544,moves=672
quick,100,random,compares=1027,moves=555
merge,100,near,compares=481,moves=672
quick,100,near,compares=1124,moves=642
merge,100,rev,compares=316,moves=672
quick,100,rev,compares=1056,moves=543
merge,500,random,compares=3838,moves=4488
quick,500,random,compares=6757,moves=4160
merge,500,near,compares=3368,moves=4488
quick,500,near,compares=7388,moves=4269
merge,500,rev,compares=2216,moves=4488
quick,500,rev,compares=6549,moves=3753
merge,1000,random,compares=8746,moves=9976
quick,1000,random,compares=13464,moves=7987
merge,1000,near,compares=7662,moves=9976
quick,1000,near,compares=16401,moves=9516
merge,1000,rev,compares=4932,moves=9976
quick,1000,rev,compares=14591,moves=8498
```

Figure 63: Screenshot of Output (OpCounts) for Module 4

```
ALGO,SIZE,COND,SECONDS
merge,100,random,0.0002812000457197428
quick,100,random,0.00034639996010810137
merge,100,near,0.00021440000273287296
quick,100,near,0.000278300023637712
merge,100,rev,0.00020210002548992634
quick,100,rev,0.0002804000396281481
merge,500,random,0.001590200001373887
quick,500,random,0.0017110999906435609
merge,500,near,0.0013837999431416392
quick,500,near,0.0017397000920027494
merge,500,rev,0.0011427999706938863
quick,500,rev,0.001496500102803111
merge,1000,random,0.003812100039795041
quick,1000,random,0.002808299963362515
merge,1000,near,0.0031986000249162316
quick,1000,near,0.003713299985975027
merge,1000,rev,0.0024529999354854226
quick,1000,rev,0.0033438999671489
```

Figure 64: Screenshot of Output (timing summary) for Module 4

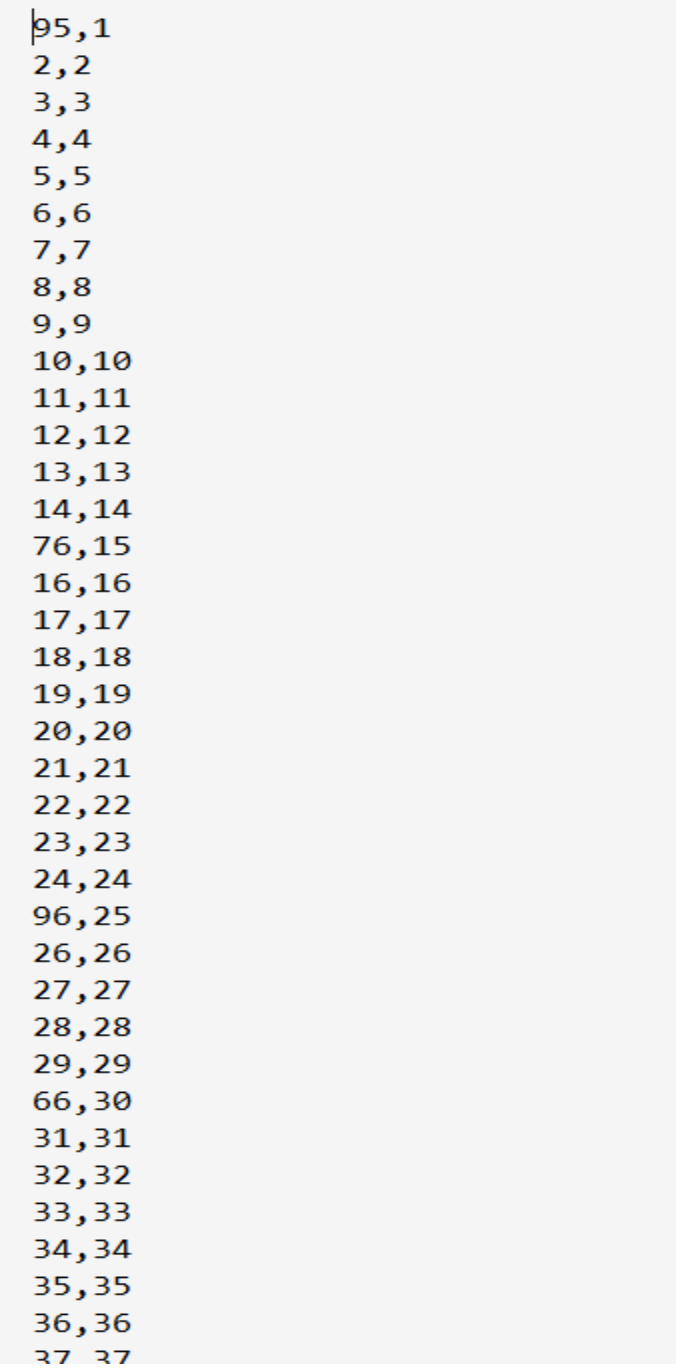

Figure 65: Screenshot of sample generated Output1 (near) for Module 4

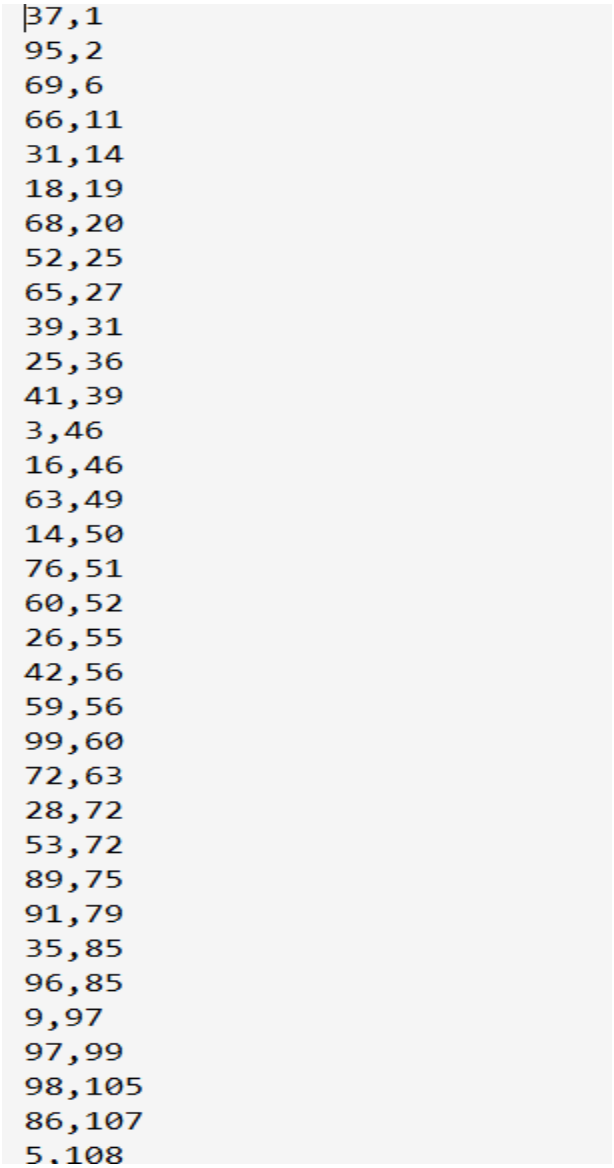

Figure 66: Screenshot of sample generated Output2 (random) for Module 4

```
100,1
99,2
98,3
97,4
96,5
95,6
94,7
93,8
92,9
91,10
90,11
89,12
88,13
87,14
86,15
85,16
84,17
83,18
82,19
81,20
80,21
79,22
78,23
77,24
76,25
75,26
74,27
73,28
72,29
71,30
70,31
69,32
68,33
67,34
66,35
65,36
```

Figure 67: Screenshot of sample generated Output3 (reverse) for Module 4

6) Reflection

The building of the hospital resource management system gave the understanding of the way the modular data structures can be related in real-life optimisation issues.

- ➢ The complements of BFS, DFS and A-star, reachability, structural analysis and efficient path finding respectively, were demonstrated in the graph module (Module 1) in terms of heuristics. The underlying internal memory structure in the form of Linked-List made it more flexible in the insertion at the expense of the pointer overhead.
- ➢ The hash table (Module 2) provided almost constant time look up of the patients, but there were trade-offs between speed and memory to strike a balance between the load factors and resizing thresholds. Chained collision handling was more stable and expensive to traverse in high loads.
- ➢ The best performing module is a heap scheduler (Module 3) in terms of dynamic priority update in case of emergencies. This design was more precise in terms of scheduling than it was easy since it had to do a set of calculations to support the heap order after an insert or update.
- ➢ The sorting module (Module 4) compared Merge Sort and Quick sort in the case of linked lists. Merge sort was more predictable and stable whereas Quick sort worked well in almost-sorted inputs. Benchmarking was used to show how data properties can be used to dictate the choice of algorithms.

A combination of the modules claimed that testability is enhanced by clean modular abstraction and that reuse is enhanced by clean modular abstraction and that the correspondence between algorithms and data structures determines efficiency in general.

7) Limitations & Assumptions

To be modular and to be able to understand the algorithms the system design actually simplified several real-world items. Every module is operating on certain assumptions and limitations that limit their generalisability as well as scalability.

<u>Module 1 - Graph (Weighted, Undirected):</u>

- ➔ The assumption is that the graph is undirected and totally symmetric i.e. there is no direction and all weights of an edge are equal.
- ➔ Co-ordinates of a node (x, y) are optional and only used where heuristic distance functions are used in A* (Manhattan metric). There was no modelling of the dynamic costs or the actual geographical distance.
- ➔ The BFS and DFS algorithms lack the ability of multiple-target or disconnected graph determination except when identifying the unreachable nodes.
- ➔ A* algorithm assumes that the heuristic will be constant, but can be worse on irregular or missing coordinate information graphs.
- ➔ The scale of the graph was small so that the graph can be read, no measurements were taken on the memory performance as well as time performance of large and dense graphs.

Module 2 - Hash-Based Patient Lookup

- ➔ Patient IDs are expected to be unique numbers, non-duplicated integer numbers.
- ➔ The hash table uses chaining to solve collisions but as the load factor exceeds the threshold, linked list length can be used to degrade performance.
- ➔ The simple prime-number resizing that is not optimal distribution is called Rehashing.
- ➔ No long term storage and information recovery, all the records are in memory when the programs are being executed.
- ➔ The concept of validation rests upon the fact that input files should be in the right format (order and spacing of tokens); any invalid entries are not counted or corrected, instead, they go to log.

Module 3 - Heap-Based Emergency Scheduler

- ➔ The scheduling priority function is simplified because (6 - UrgencyLevel) + (1000 / treatment time) considers the urgency and the time as factors of equal importance. Dynamic weighting would need to be implemented in real-world triage in a hospital.
- ➔ The requests are handled in a sequence with the assumption that there would be no limit in medical staff and also there would be no external limit in terms of beds or doctors.
- ➔ The heap does not provide a decrease-key, but rather a new request with a new priority should be inserted.

- ➔ The patient hash table presupposes that all patients have been previously loaded, absent or invalid IDs in requests are logged, but not restored dynamically.
- ➔ Operations are executed based on the calculation of priority only and not on arrival time or fairness schedule.

Module 4 - Arranging Patient Records

- ➔ Datasets that were sorted were artificial, typically with a few thousand entries and were based on runtime and counts of operations as opposed to data persistence.
- ➔ Just the linked-list versions of Merge Sort and Quick Sort were tested; there were no array based versions or hybrid sorts.
- ➔ All treatment times are assumed to be positive integers, invalid and missing treated as null.
- ➔ The timing measurements are based on the perf-counter() of Python without considering the system noise, that is why the timing measurements could have some variation between runs.
- ➔ Output messages were stored as plain text, without any database or other API coupling.

General System-Wide Limitations.

- ☐ All the modules are based on a custom linked list structure, which is easy to insert and delete but consumes more time to traverse to access a random element.
- ☐ It is single threaded and non persistent implementation there is no concurrency or database connectivity.
- ☐ Error handling is concerned with stability as opposed to user interactivity- logs are text based and the interface makes the assumption of correct input filenames.
- ☐ The entire architecture is focused on education, comparison of algorithms and traceability, rather than on robustness or scalability on an enterprise-wide basis.

Reference

Hart, P., Nilsson, N., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100–107. https://doi.org/10.1109/tssc.1968.300136